THE UNIVERSITY OF AZAD JAMMU & KASHMIR

DEPARTMENT OF SOFTWARE ENGINEERING

Course Title: Information Security

Course Code: CS-4101

Instructor: Engr. Dr Ghulam Sarwar

Semester: Fall-2024

Session: 2021 – 25

Assignment#01

By

EESHA KHAN 2021-SE-01

Submission date: 12th, Nov, 2024

Instructor Signature: _____

Question#1

Explain how the CORDIC algorithm can be applied in the context of information security. Identify one specific example of how its use in trigonometric or hyperbolic functions can enhance encryption or data protection measures.

Assessment Criteria:

- Demonstrates understanding of the CORDIC algorithm's functionality in processing trigonometric and hyperbolic functions.
- Recognizes an application in the information security domain, such as in cryptographic algorithms, key generation, or secure signal processing.

Answer

The CORDIC (Coordinate Rotation Digital Computer) algorithm is an efficient, iterative technique for calculating trigonometric, hyperbolic, exponential, and logarithmic functions. Originally developed for computer systems with limited hardware resources, it approximates complex functions using a series of simple shifts, additions, and lookup tables, allowing calculations with minimal computational power and without the need for multiplication or division. In the context of information security, CORDIC's capability to process trigonometric and hyperbolic functions efficiently offers several advantages, particularly in cryptographic applications where speed, efficiency, and accuracy are crucial.

Applying CORDIC in Information Security

CORDIC's use in information security typically centers on its efficiency in processing trigonometric and hyperbolic functions for tasks like encryption, key generation, and secure signal processing. Cryptographic algorithms often rely on functions that can be computationally intensive, especially when large datasets or complex functions are involved. CORDIC's ability to compute trigonometric values efficiently makes it highly suitable for secure, embedded systems where resources are limited, such as IoT devices and smart cards.

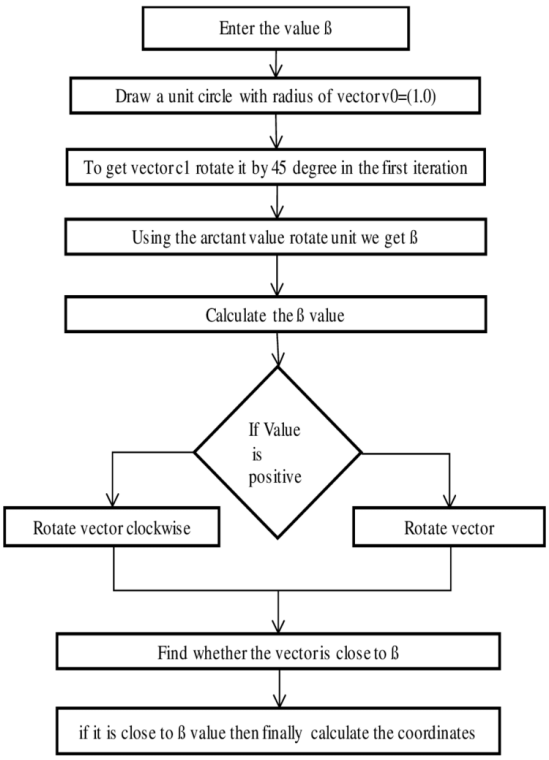

Figure 1

Example

Application: Enhanced Key Generation with Elliptic Curve Cryptography (ECC)

One specific application of the CORDIC algorithm in information security is in Elliptic Curve Cryptography (ECC), a cryptographic technique widely used in secure communications due to its high security with smaller key sizes than traditional algorithms. ECC relies on the properties of elliptic curves, which involve trigonometric calculations for point additions and scalar multiplications—operations critical for generating public and private keys.

How CORDIC Enhances ECC:

1. **Efficient Computation:** CORDIC enables faster and more resource-efficient calculations of the trigonometric and hyperbolic functions required for ECC, such as angle rotations in point transformations. This helps perform ECC calculations with limited hardware, making ECC viable for devices with restricted processing power and memory.
2. **Low Latency:** The iterative approach of CORDIC minimizes computation time, reducing latency in secure data transactions. This low-latency feature is especially beneficial in

high-speed encryption and decryption processes, enhancing the performance of ECC-based systems.

3. **Reduced Power Consumption:** Because CORDIC eliminates the need for multiplication, ECC implementations on devices like smart cards or mobile hardware benefit from lower power requirements. This improves the longevity and reliability of security protocols in mobile and battery-powered environments.

Benefits of Encryption and Data Protection

- **Faster and More Secure Key Generation:** ECC with CORDIC-enhanced computations allows for faster key generation and encryption, improving the overall security by making it harder for attackers to exploit timing vulnerabilities.
- **Increased Feasibility on Resource-Constrained Devices:** CORDIC makes ECC practical for many devices, especially IoT devices that require high security but lack substantial processing resources.

Problem Statement

Suppose we need to perform point rotations as part of an elliptic curve encryption process. Let's say that for a point (x_0, y_0) on the elliptic curve, we need to rotate it by an angle θ to generate another point (x, y), which will be used in key generation.

For simplicity, let's assume:

- $x_0 = 1$
- $y_0 = 0$
- Rotation angle $\theta = 45^\circ$

The rotation formulas are:

$$x = x_0 \cdot \cos(\theta) - y_0 \cdot \sin(\theta)$$

$$y = x_0 \cdot \sin(\theta) + y_0 \cdot \cos(\theta)$$

Using the **CORDIC algorithm**, we'll find $\cos(45^\circ)$ and $\sin(45^\circ)$ iteratively.

Step-by-Step Solution Using CORDIC

Step 1: Initialize Values

CORDIC calculates rotations by approximating them as a series of micro-rotations. The algorithm uses the following settings:

- A lookup table of predefined angles $\arctan(2^{-i})$.
- Initial point coordinates $(x_0, y_0) = (1, 0)$.
- Accumulated rotation angle $z = 45^\circ$ (or $\frac{\pi}{4}$ radians).

For simplicity, we'll assume we're working with three iterations, though real applications use more for higher precision.

Step 2: Iterative Rotations

CORDIC performs rotations iteratively with these formulas:

$$x_{i+1} = x_i - d_i \cdot y_i \cdot 2^{-i}$$

$$y_{i+1} = y_i + d_i \cdot x_i \cdot 2^{-i}$$

$$z_{i+1} = z_i - d_i \cdot \arctan(2^{-i})$$

where d_i is the direction of rotation, determined by the sign of z:

- If $z > 0$, $d_i = +1$
- If $z < 0$, $d_i = -1$

Iteration Calculations

1. **Iteration 1** (i = 0, $\arctan(2^{-0}) = 45^\circ$):
   - $d_0 = +1$ (since $z = 45^\circ > 0$)
   - $x_1 = x_0 - d_0 \cdot y_0 \cdot 2^0 = 1 - 0 = 1$
   - $y_1 = y_0 + d_0 \cdot x_0 \cdot 2^0 = 0 + 1 = 1$
   - $z_1 = z_0 - d_0 \cdot \arctan(1) = 45^\circ - 45^\circ = 0^\circ$
2. **Iteration 2** (i = 1, $\arctan(2^{-1}) \approx 26.57^\circ$):
   - $d_1 = -1$ (since $z = 0 < 26.57^\circ$)
   - $x_2 = x_1 - d_1 \cdot y_1 \cdot 2^{-1} = 1 - (-1) \cdot 1 \cdot 0.5 = 1 + 0.5 = 1.5$
   - $y_2 = y_1 + d_1 \cdot x_1 \cdot 2^{-1} = 1 + (-1) \cdot 1 \cdot 0.5 = 1 - 0.5 = 0.5$
   - $z_2 = z_1 - d_1 \cdot \arctan(0.5) \approx 0^\circ - (-26.57^\circ) = 26.57^\circ$
3. **Iteration 3** (i = 2, $\arctan(2^{-2}) \approx 14.04^\circ$):
   - $d_2 = +1$ (since $z = 26.57^\circ > 14.04^\circ$)
   - $x_3 = x_2 - d_2 \cdot y_2 \cdot 2^{-2} = 1.5 - 1 \cdot 0.5 \cdot 0.25 = 1.5 - 0.125 = 1.375$
   - $y_3 = y_2 + d_2 \cdot x_2 \cdot 2^{-2} = 0.5 + 1 \cdot 1.5 \cdot 0.25 = 0.5 + 0.375 = 0.875$
   - $z_3 = z_2 - d_2 \cdot \arctan(0.25) \approx 26.57^\circ - 14.04^\circ = 12.53^\circ$

Final Approximation

After these three iterations, we have an approximate rotated point $(x_3, y_3) = (1.375, 0.875)$, which is close to the expected values for $\cos(45^\circ)$ and $\sin(45^\circ)$ scaled by the initial vector length.

Summary

CORDIC's role in trigonometric and hyperbolic computations makes it an excellent choice for enhancing cryptographic applications like ECC. By streamlining these calculations, CORDIC contributes to faster, more efficient encryption processes and key generation, ultimately strengthening the security measures in resource-constrained environments.

Question#02

Describe the role of the AES algorithm in ensuring information security. Explain how its symmetric key structure contributes to data confidentiality in applications such as secure communications or file encryption.

Assessment Criteria:

- Shows understanding of the AES algorithm's purpose and basic operation.
- Recognizes the role of symmetric key encryption in maintaining data confidentiality.
- Provides a real-world application of AES in securing sensitive data.

Answer

The **Advanced Encryption Standard (AES)** algorithm is essential for safeguarding sensitive data in digital communications, file encryption, and other applications that require secure data handling. AES helps ensure data confidentiality, meaning it keeps data private by transforming it into a format that only authorized users can access.

Purpose and Operation of AES

AES is a symmetric key encryption algorithm, meaning it uses a single key for both encrypting and decrypting data. AES encrypts data in blocks of 128 bits, processing it through multiple

rounds of substitutions, shifts, and mathematical operations, with each round making the data increasingly difficult to decipher without the key. The strength of AES is largely due to its key sizes—128, 192, or 256 bits—and the number of encryption rounds increases with longer keys, providing even higher security.

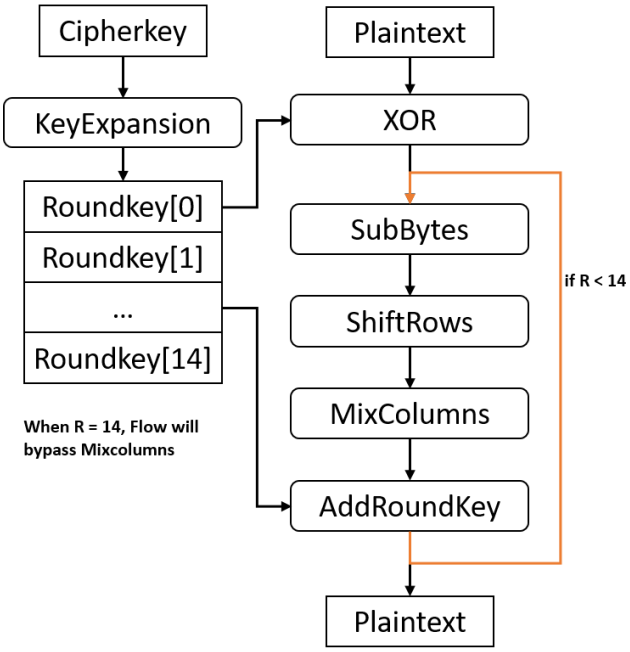

Figure 2

Role of Symmetric Key Structure in Data Confidentiality

In AES, the symmetric key structure ensures that data is only accessible to users who possess the encryption key. Since the same key is used for both encryption and decryption, maintaining the confidentiality of the key itself is critical. When data is encrypted with AES, anyone without the key will see only unreadable information, protecting it from unauthorized access. This makes AES particularly suitable for secure, high-speed encryption in scenarios where the same key can be securely shared, such as in private messaging, file storage, or VPNs.

Real-World Application of AES

One practical example of AES in use is secure web browsing over HTTPS, where AES encryption protects data transmitted between a user's browser and a website. This ensures that sensitive information like passwords, financial details, and personal information cannot be accessed by hackers intercepting the communication. AES is also used in file encryption tools

like BitLocker and File Vault, which protect data on devices by encrypting it so only authorized users can decrypt and view it. In both cases, AES's symmetric key structure provides a fast, reliable way to secure sensitive data.

Example

- P = Plaintext (input data, 128 bits)
- C = Ciphertext (encrypted output)
- K = 128-bit key (used for encryption and decryption)
- Round Key_i = Key schedule for round i (generated from K)
- $\oplus$ = XOR operation
- **S-Box**, **ShiftRows**, **MixColumns** = Mathematical operations applied during encryption

AES Encryption (AES-128):

1. **Key Expansion:** Generate round keys from the original key K.
2. **Initial Round:**

$$P_0 = P \oplus \text{Round Key}_0$$

3. **Rounds (1 to 9):** For each round i (1 to 9):

$$P_i = \text{SubBytes}(\text{ShiftRows}(\text{MixColumns}(P_{i-1}))) \oplus \text{Round Key}_i$$

4. **Final Round (Round 10):**

$$C = \text{SubBytes}(\text{ShiftRows}(P_9)) \oplus \text{Round Key}_{10}$$

The result is the ciphertext C.

AES Decryption (AES-128):

1. **Key Expansion:** Generate round keys from the original key K.
2. **Initial Round:**

$$C_0 = C \oplus \text{Round Key}_{10}$$

3. **Rounds (1 to 9):** For each round i (1 to 9):

$$C_i = \text{InverseMixColumns}(\text{InverseShiftRows}(\text{InverseSubBytes}(C_{i-1}))) \oplus \text{Round Key}_{10-i}$$

4. **Final Round (Round 0):**

$$P = \text{InverseShiftRows}(\text{InverseSubBytes}(C_9)) \oplus \text{Round Key}_0$$

The result is the plaintext P.

Conclusion

In summary, AES is a cornerstone of data security for many modern applications. Its symmetric key structure, while requiring secure key management, provides a highly effective means of protecting data confidentiality across various digital platforms, ensuring that only authorized users can access the data.